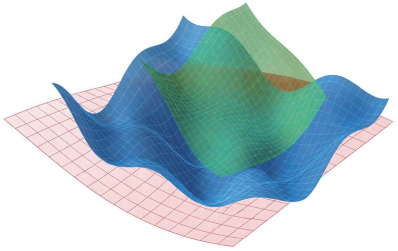
으뜸 머신러닝

Chapter 02

머신러닝을 위한 기초지식

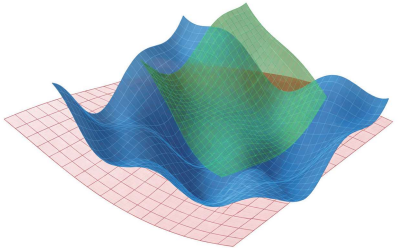


CHAPTER

02 머신러닝을 위한 기초지식

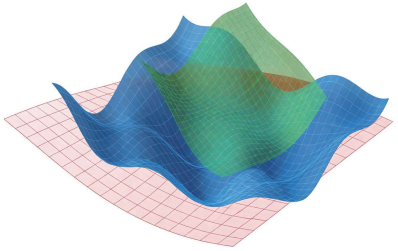
CONTENTS

2.1 수학 표기
2.2 벡터와 행렬
2.3 벡터와 행렬의 기본 연산
2.4 미분과 기울기, 그리고 경사하강법의 개념
2.5 편미분과 기울기
2.6 모델, 파라미터, 그리고 학습
2.7 오차의 기울기를 이용한 학습의 기본 원리와 최적해
2.8 과적합, 과소적합, 그리고 일반화
2.9 과적합을 피하기 위한 방법들



이 장에서 생각할 문제

- 모델, 파라미터, 학습은 머신러닝에서 어떻게 작동하는가?
- 이들 요소가 상호작용하여 어떤 결과를 만들어내는가?
- 파라미터는 모델의 어떤 부분에 영향을 미치는가?
- 오차란 무엇이며, 이를 이용해 어떻게 학습을 진행할 수 있는가?
- 오차를 줄이는 과정에서 어떤 수학적 원리가 적용되는가?
- 학습 과정은 무엇을 최적화하며, 하이퍼파라미터는 어떤 역할을 하는가?
- 하이퍼파라미터는 학습 과정에서 어떤 점을 조절하는가?
- 최적화 과정에서 하이퍼파라미터가 학습 결과에 미치는 영향은 무엇인가?



2.1 수학 표기

- **선형 대수학**linear algebra과 미적분의 개념은 머신러닝을 설계하고 이해하는 데에 필요함.
- 정확한 개념 전달과 구현을 위해 모호하지 않은 수학적 표기 약속이 필요함.

Αα...Ωω

그리스 문자			
Αα	알파	Νν	뉴
Ββ	베타	Ξξ	크시
Γγ	감마	Οο	오미크론
Δδ	델타	Ππ	파이
Εε	엡실론	Ρρ	로
Ζζ	제타	Σσς	시그마
Ηη	에타	Ττ	타우
Θθ	세타	Υυ	입실론
Ιι	요타	Φφ	피
Κκ	카파	Χχ	키
Λλ	람다	Ψψ	프시
Μμ	뮤	Ωω	오메가
기타 문자			
ϝ	디감마	ϛ	스티그마
ϟ	헤타	ϡ	산
ϙ	코파	ϡ	삼피
ϸ	쇼		

수, 벡터, 행렬

표기법	의미	특징 및 주의
a	스칼라 변수	보통 굵기의 이탤릭체
A	스칼라 상수	대문자
$\mathbf{a}$	벡터	굵은 정자체 소문자
$\mathbf{A}$	행렬	굵은 정자체 대문자
$\mathbf{I}_n$	$n \times n$ 차원 항등행렬	아래 첨자로 차원 표기

집합

표기법	의미	특징 및 주의
$\mathbb{R}$	실수 집합	대문자 외곽선
$\mathbb{R}^n$	n 차원 실수 벡터 집합	윗 첨자로 벡터의 차원 표기
$\mathbb{R}^{n \times m}$	n 행 m 열의 행렬 집합	윗 첨자로 행과 열의 수 표기
$\in$	원소	왼쪽이 오른쪽 집합의 원소
$\mathbf{A} \in \mathbb{R}^{n \times m}$	$\mathbf{A}$ 는 n 행 m 열의 행렬	$\mathbf{A}$ 가 $\mathbb{R}^{n \times m}$ 의 원소

2.1 수학 표기

선형 대수

표기법	의미	특징 및 주의
$\mathbf{A}^{\mathrm{T}}$	행렬 $\mathbf{A}$의 전치	첨자 T로 전치 표현
$\mathbf{A}^{-1}$	행렬 $\mathbf{A}$의 역행렬	첨자 −1로 곱셈에 대한 역원 표현
$\mathrm{tr}(\mathbf{A})$	행렬 $\mathbf{A}$의 **대각성분 합**[trace]	함수 표현
$\mathbf{A} \otimes \mathbf{B}$	**아다마르**[Hadamar] 곱	연산자에 원을 씌어 원소별 연산 표현

미적분

표기법	의미	특징 및 주의
$f : \mathbb{R}^n \to \mathbb{R}^m$	함수	$\mathbb{R}^n$ 에 속한 값을 $\mathbb{R}^m$ 으로 옮기는 함수
f'	함수 f의 미분	어깨점으로 미분 표현
$\frac{df}{dx}$	함수 f의 x에 대한 미분	미분의 변수를 명시적으로 표현
$f'(a)$	a 위치에서 함수 f의 미분	특정 지점에서의 미분, $f : \mathbb{R} \to \mathbb{R}^*$
$\frac{\partial f}{\partial x_i}$	다차원 입력 함수 f의 x_i 에 대한 편미분	x_i를 제외한 모든 입력 변수를 상수로 취급 $f : \mathbb{R}^n \to \mathbb{R}^*$
$\frac{\partial f}{\partial x_i}(\mathbf{a})$	입력 벡터 $\mathbf{a}$ 위치에서 함수 f의 x_i 에 대한 편미분	입력 벡터 $\mathbf{a}$ 위치에서 x_i를 제외한 모든 변수를 상수로 취급하여 미분
∇f	함수 f의 기울기	$f : \mathbb{R}^n \to \mathbb{R}$ 의 기울기
$\nabla_a f$	함수 f의 기울기	벡터 a에 대한 함수 f의 기울기

2.2 벡터와 행렬

- **머신러닝과 선형대수의 기본 개념**
  - 머신러닝은 다차원 특징 데이터를 입력으로 받아 분류·판단을 수행함
  - 이러한 데이터는 일반적으로 벡터로 표현되며, 변환 과정은 행렬 곱으로 나타냄
  - 벡터와 행렬의 성질·연산을 다루는 학문이 선형대수
  - 스칼라는 하나의 크기 값을 가지는 수
  - 벡터는 순서가 있는 스칼라 배열이며, 보통 열벡터로 표현함
  - 열벡터는 전치를 통해 행벡터로 변환 가능함

$$\mathbf{x} = \begin{vmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{vmatrix} \qquad \mathbf{x}^{\mathrm{T}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$$

2.2 벡터와 행렬

- n 차원 벡터 **x**는 R^n 의 원소로 다음과 같이 표현함.

$$\mathbf{x} \in \mathbb{R}^n$$

- 벡터는 크기와 방향을 갖는 값인데, 벡터의 크기를 벡터의 **노름**norm이라고 함.
- 벡터 **x**의 p차 노름의 정의

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{\frac{1}{p}}$$

- p =1이라면 각 차원별 거리의 합인 **맨해튼 거리**Manhatten distance이고, l_1 노름이라 함.
- p =2 이면 벡터의 시작점에서 끝점을 화살표로 표현할 때 화살표의 길이와 같은 **유클리드 노름**Euclidian norm이며, l_2 노름이라 함.
- p를 생략하고 노름을 사용할 경우 일반적으로 p =2 인 l_2 노름을 말함.

노름의 성질
1. 양수성
   $\|x\| \geq 0$
2. 영벡터의 조건
   $\|x\|=0$이면 $x=0$
3. 삼각 부등식
   $\|x+y\| \leq \|x\| + \|y\|$

벡터

$$x = (3, 4)$$

이면

$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

$$\|x\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

즉 벡터 (3,4)의 길이는 5입니다.

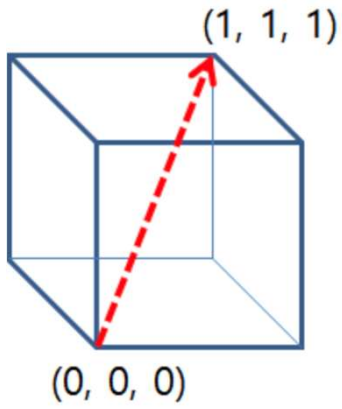
유클리드 노름

- 유클리드 노름은원점(0,0,...,0)에서 벡터 끝점까지의 거리입니다.
  - 2차원 벡터 예
  - x=(3,4)

$$\|x\| = \sqrt{3^2 + 4^2}$$
$$= \sqrt{9 + 16} = \sqrt{25} = 5$$

  - 3차원 벡터 예
  - x=(1,2,2)

$$\|x\| = \sqrt{1^2 + 2^2 + 2^2}$$
$$= \sqrt{1 + 4 + 4} = \sqrt{9} = 3$$



  - **Euclidean norm**(유클리드 노름)은 벡터의 **길이(크기)**를 나타내는 가장 기본적인 노름입니다. 보통 우리가 기하에서 사용하는 **피타고라스 방식의 거리**와 동일합니다

2.2 벡터와 행렬

- **단위 벡터**unit vector : 길이가 1인 벡터. 벡터를 유클리드 노름의 값으로 나누면 얻을 수 있음.
- **정규화**normalization : 어떤 벡터를 길이 1인 벡터로 만드는 일
- $\mathbf{x}$를 단위 벡터 $\tilde{\mathbf{x}}$로 만드는 정규화

$$\tilde{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}$$

- 벡터 $\mathbf{x}$는 크기 $\|\mathbf{x}\|_2$와 방향 $\tilde{\mathbf{x}}$의 곱으로 표현할 수 있다.

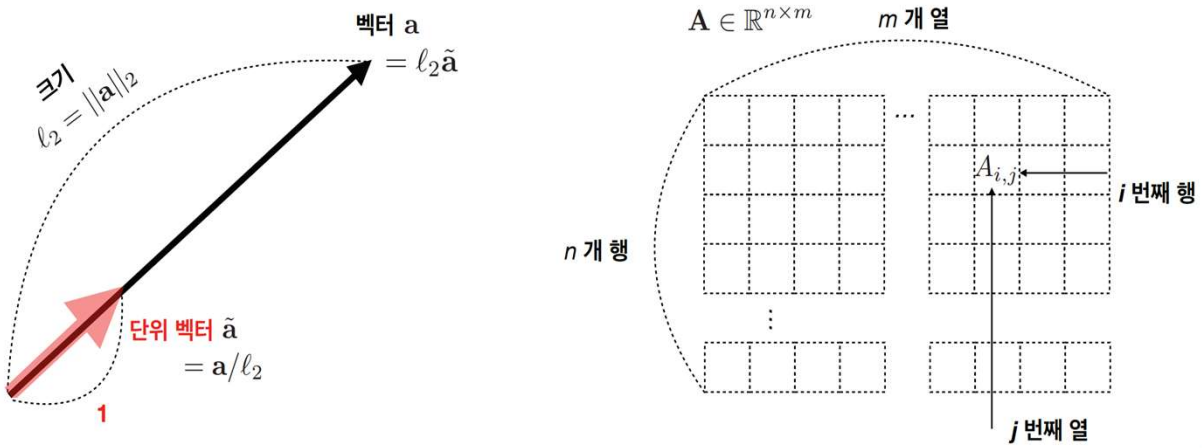
$$\mathbf{x} = \|\mathbf{x}\|_2 \cdot \tilde{\mathbf{x}} = l_2 \times \text{방향}$$

2.2 벡터와 행렬

- **행렬**matrix : 스칼라 값이나 자료를 **행**row과 **열**column로 이루어진 배열의 형태로 나타낸 것.
  - 행의 수가 n이고 열의 수가 m개인 행렬을 $n \times m$행렬이라고 함.
- $n \times m$행렬 $\mathbf{A}$를 다음과 같이 표현함.

$$\mathbf{A} \in \mathbb{R}^{n \times m}$$

- 행렬의 i행 j열 원소는 $\mathbf{A}_{i,j}$로 표현하고, 하나의 스칼라 값이 됨.
- 벡터는 [그림 2-1]의 왼쪽과 같은 화살표로, 행렬은 오른쪽과 같은 2차원 배열 형태로 종종 가시화함.



[그림 2-1] 벡터와 행렬 개념의 가시화

2.3 벡터와 행렬의 기본 연산

- 두 벡터 a와 b가 모두 동일 차원의 벡터로 R^n에 속한다고 할 때, 두 벡터를 더하거나 빼서 얻는 새로운 벡터 c는 다음과 같이 구할 수 있음.

$$\mathbf{c} = \mathbf{a} \pm \mathbf{b} \Leftrightarrow c_i = a_i \pm b_i$$

- 두 벡터의 곱은 여러 가지로 정의할 수 있는데, 가장 간단하면서 벡터 연산에서 여러 영역에서 유용한 **아다마르**Hadamard 곱은 다음과 같이 정의함. 벡터 덧셈과 뺄셈과 동일한 방식으로 원소별 곱을 구하여 새로운 벡터를 만듦.

$$\mathbf{c} = \mathbf{a} \otimes \mathbf{b} \Leftrightarrow c_i = a_i b_i$$

- 벡터의 곱으로 흔히 사용되는 **점곱**dot product과 **가위곱**cross product는 각각 스칼라곱, 벡터곱이 라고 부르는데, 그 결과가 점곱은 스칼라값, 가위곱은 벡터가 나옴.

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_1 b_1 = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^{n} (\mathbf{a} \otimes \mathbf{b})_i$$

벡터 곱

- 내적(Dot Product)
  - 두 벡터를 곱해서 **하나의 숫자(스칼라)**가 나오는 곱
  - a=(a1,a2,a3),b=(b1,b2,b3)

$$a \cdot b = a_1 b_1 + a_2 b_2 + a_3 b_3$$

$$a = (1, 2, 3), \quad b = (4, 5, 6)$$

$$a \cdot b = 1 \times 4 + 2 \times 5 + 3 \times 6$$

$$= 4 + 10 + 18 = 32$$

- 외적 (Cross Product)
  - 두 벡터를 곱해서 **새로운 벡터**가 나오는 연산

$$a \times b = (a_2 b_3 - a_3 b_2,\ a_3 b_1 - a_1 b_3,\ a_1 b_2 - a_2 b_1)$$

$$a = (1, 0, 0), \quad b = (0, 1, 0)$$

$$a \times b = (0, 0, 1)$$

2.3 벡터와 행렬의 기본 연산

- 점곱이 갖는 기하적 특성 : 두 벡터가 이루는 사잇각 α 의 **코사인**cosine 값에 비례하고, 두 벡터의 크기에 각각 비례함.

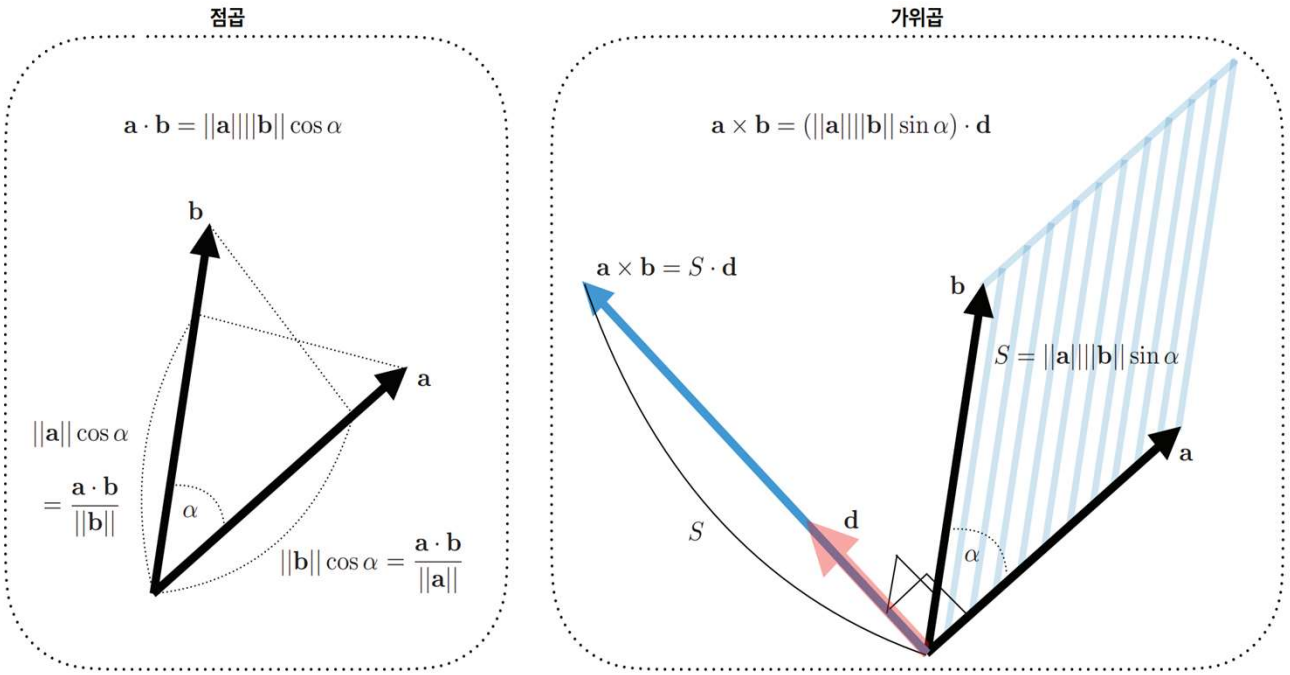
$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \alpha$$

- 가위곱은 벡터곱이므로 크기와 방향을 갖는 새로운 벡터를 얻게 되는데, 크기는 두 벡터의 크기와 사잇각의 **사인**sine 값에 비례하고, 방향은 두 벡터에 동시에 수직인 방향임.
  - 단, $\mathbf{d}$는 두 벡터에 동시에 주식인 방향의 단위 벡터

$$\mathbf{a} \times \mathbf{b} = (\|\mathbf{a}\| \|\mathbf{b}\| \sin \alpha) \cdot \mathbf{d}$$

2.3 벡터와 행렬의 기본 연산

- 점곱은 두 벡터가 같은 방향일 때 큰 값을 얻고, 수직이면 0, 반대 방향 을 향하면 음수가 됨.
- 가위곱은 두 벡터가 수직일 때 최대의 크기를 갖고, 같은 방향이나 반대방향이 되면 크기가 0이 됨.
- 점곱과 가위곱의 기하적 의미



[그림 2-2] 두 벡터의 점곱(내적)과 가위곱(벡터곱)의 가시화

2.3 벡터와 행렬의 기본 연산

- 두 행렬 **A**와 **B**를 곱할 때는 **A**의 열의 개수와 **B**의 행의 개수가 일치해야 한다. 따라서 다음과 같은 경우에 곱이 가능함.

$$\mathbf{A} \in \mathbb{R}^{n \times m}$$
$$\mathbf{B} \in \mathbb{R}^{m \times l}$$

- 두 행렬의 곱 **C**는 $\mathbb{R}^{n \times l}$ 집합에 속하며 각 원소는 다음과 같이 계산함.

$$\mathbf{C} = \mathbf{AB}$$
$$C_{i,j} = \sum_{k=1}^{m} A_{i,k} B_{k,j} = A_{i,1} B_{1,j} + A_{i,2} B_{2,j} + A_{i,3} B_{3,j} + \cdots + A_{i,m} B_{m,j}$$

2.3 벡터와 행렬의 기본 연산

- 행렬 $\mathbf{A}$의 i 행 벡터를 $\mathbf{A}_{i,*}$라고 하고, $\mathbf{B}$의 j 열 벡터를 $\mathbf{B}_{*,j}$ 라고 하면, 행렬의 곱 $\mathbf{C}$의 i행 j열 원소는 다음과 같이 두 벡터의 점곱임을 알 수 있다.

$$C_{i,j} = \mathbf{A}_{i,*}\mathbf{B}_{*,j} = \mathbf{A}_{i,*}^T \cdot \mathbf{B}_{*,j}$$

$$C_{i,j} = \begin{bmatrix} A_{i,1} & A_{i,2} & \cdots & A_{i,m} \end{bmatrix} \begin{bmatrix} B_{1,j} \\ B_{2,j} \\ \vdots \\ B_{m,j} \end{bmatrix} = \begin{bmatrix} A_{i,1} \\ A_{i,2} \\ \vdots \\ A_{i,m} \end{bmatrix} \cdot \begin{bmatrix} B_{1,j} \\ B_{2,j} \\ \vdots \\ B_{m,j} \end{bmatrix}$$

- 두 벡터 $\mathbf{a}$와 $\mathbf{b}$의 점곱을 표현할 때 벡터를 1개 열을 가진 행렬로 보면 다음과 같은 행렬곱 표현으로 나타낼 수 있다.

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b}$$

2.4 미분과 기울기, 그리고 경사하강법의 개념

- 어떤 값이 어떻게 변하는지를 정확하게 알기 위해서

1. 변화의 속도를 알기 위해(Calculus)
   - 미분은 **변화율**을 구하는 방법입니다.
   - 예를 들어 **시간에 따른 위치 변화**를 미분하면 **속도**가 됩니다.
     - 위치 → 미분 → 속도
     - 속도 → 미분 → 가속도
2. 최대값과 최소값을 찾기 위해(Optimization)
   - 미분을 사용하면 함수의 최대값과 최소값을 찾을 수 있습니다.
   - 예
     - 회사 이익의 최대값
     - 비용의 최소값
     - 생산량 최적화
3. 그래프의 기울기를 구하기 위해
   - 미분은 그래프에서 **한 점에서의 기울기**를 나타냅니다.
   - 함수 $y=x^2$ 미분하면 $\frac{dy}{dx} = 2x$

2.4 미분과 기울기, 그리고 경사하강법의 개념

- **미분**derivative : 순간 변화량을 구하는 것. 독립 변수값의 변화량의 비의 극한으로 구성됨.
- $y = f(x)$와 같은 함수의 한 입력값 a에서의 미분 $f'(a)$의 정의

$$f'(a) = \lim_{\Delta x \to 0} \frac{f(a + \Delta x) - f(a)}{\Delta x}$$

- 함수 $f(x), g(x)$가 있고 상수 C가 있을 때 x에 대한 미분의 기본 공식

$\frac{d}{dx}(C) = 0$	$\frac{d}{dx}[Cf(x)] = Cf'(x)$
$\frac{d}{dx}x^n = nx^{n-1}$	$\frac{d}{dx}e^x = e^x$
$\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$	$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$

2.4 미분과 기울기, 그리고 경사하강법의 개념

- **연쇄법칙**chain rule : 어떤 함수 y를 x에 대해 미분할 때, 매개변수 t를 두어 다음과 같이 미분할 수 있다.

$$\frac{dy}{dx} = \frac{dy}{dt}\frac{dt}{dx}$$

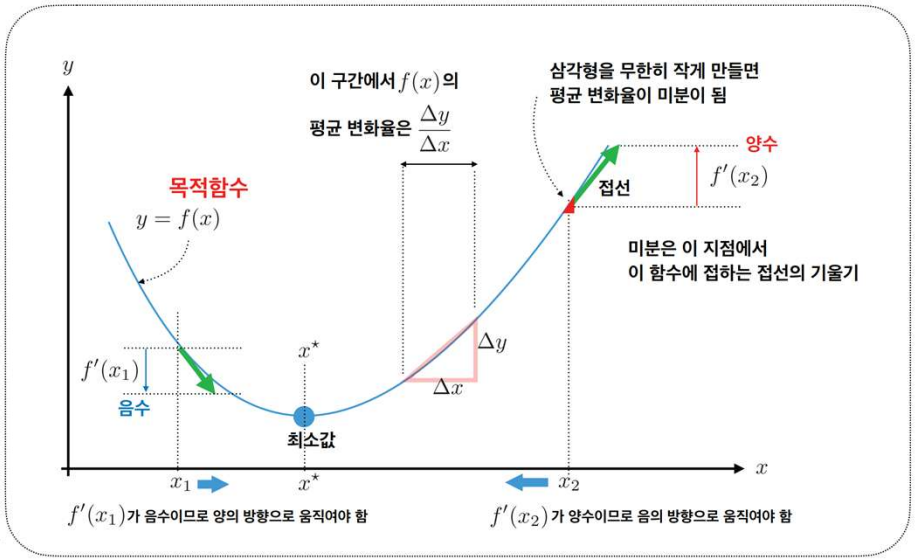
- $y = f(g(x))$와 같은 합성함수에 대한 미분을 할 때 사용된다. $t = g(x), y = f(t)$로 두고 다음과 같이 미분을 수행하여 기본 공식 테이블과 같은 결과를 얻을 수 있다.

$$\frac{d}{dx}[f(g(x))] = \frac{d}{dg(x)}f(g(x))\,\frac{d}{dx}g(x) = f'(g(x))g'(x)$$

- **최적화**optimization : 목적 함수를 f(x)라고 할 때, 이 함수를 가장 작은 값으로 만드는 최적의 변수 x^*를 찾는 일. 머신러닝에서 매우 중요하다.
- $f(x)$의 1차 미분 $f'(x)$는 x가 매우 조금 변화한 정도에 대해 함수값이 어떤 비로 변화하는지 알려준다. 이러한 성질을 이용하면 최적화 작업을 할 수 있다.

2.4 미분과 기울기, 그리고 경사하강법의 개념

- **최적화 방법**
  - 평균 변화율: 붉은 삼각형에서 변수 x의 변화량 Δx와 목적함수의 변화량 Δy의 비.
  - 미분과 접선의 기울기: $x = x_2$인 지점에서 이 삼각형을 매우 작게 만들었다. 이 삼각형을 무한히 작게 만들면 이것이 바로 x_2지점에서의 목적함수 미분이다. 이것은 접선의 기울기이고, 경사가 급할수록 더 큰 값이 된다.
  - 경사하강: 접선의 기울기를 따라 반대로 내려오면 해당 지점에서 목적함수의 값이 더 작은 쪽으로 이동할 수 있다. x_2의 위치에서 접선의 기울기는 양수이다. 따라서 x_2에서 음의 방향으로 움직이면 목적함수를 줄일 수 있다. 즉 $f'(x_2)$의 반대 방향인 $-f'(x_2)$로 이동한다. 이것을 **경사 하강법**gradient descent이라고 한다. x_1의 지점에서 미분은 음수임을 확인할 수 있다. 따라서 이 위치에서는 양의 방향으로 이동한다.



[그림 2-3] 기울기를 따라 해를 찾는 경사하강법 개념

최적화

1. 미분을 사용하는 방법
   1. 함수 $f(x)$f(x)를 미분한다.
   2. $f'(x)$=0f'(x)=0인 지점을 찾는다.
   3. 그 지점이 최대값인지 최소값인지 확인한다.

$$f(x) = x^2 - 4x + 3$$

$$f'(x) = 2x - 4$$

$$2x - 4 = 0$$

$$x = 2$$

2. 경사 하강법
   1. 기울기가 내려가는 방향으로 이동
   2. 반복하면서 최소값에 가까워짐

$$x_{new} = x_{old} - \alpha \frac{df(x)}{dx}$$

- x_{old} : 현재 값
- x_{new} : 업데이트된 값
- α : 학습률(learning rate)
- $\frac{df(x)}{dx}$: 미분값

2.5 편미분과 기울기

- **편미분**partial derivative : 둘 이상의 변수들을 가지는 함수 f가 있을 경우, 이 함수를 각각의 변수에 대해서 독립적으로 미분을 하는 방식. 하나에 대해 미분하고 나머지는 상수로 취급하는 방법.
- x, y 두 변수로 이루어진 함수 $f(x, y) = x^2 + xy + y^2$가 있을 경우 x, y에 대한 편미분은 다음과 같이 구한다.

$$\frac{\partial f}{\partial x}(x, y) = 2x + y, \quad \frac{\partial f}{\partial y}(x, y) = x + 2y$$

- 다음과 같이 복잡한 함수도 x에 대한 편미분을 하면 x가 포함된 항만 미분하여 간단히 구할 수 있다.

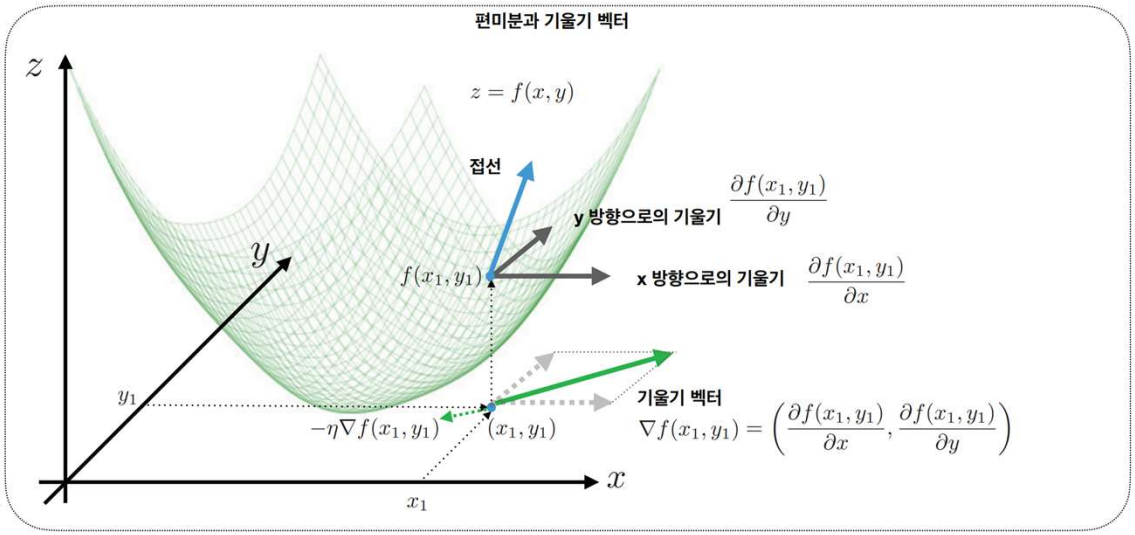
$$f(x, y, z) = x^2 + \sin(yz) + \log(z\cos(y))$$

$$\frac{\partial f(x, y, z)}{\partial x} = 2x$$

2.5 편미분과 기울기

- [그림 2-4]와 같이 독립 변수 x와 y에 의해 결정되는 목적함수 $f(x, y)$가 녹색선으로 표시된 곡면을 이룬다고 하자.
- 그림에서 $\frac{\partial f(x_1,y_1)}{\partial x}$는 목적함수의 $f(x_1, y_1)$에 닿는 접선이 x축 방향으로 갖는 기울기를 의미한다. 비슷하게 $\frac{\partial f(x_1,y_1)}{\partial y}$는 y축 방향 기울기이다.
- $x \in R^n$의 벡터를 입력으로 하는 함수 $f(\mathbf{x})$의 기울기 벡터 $\nabla f(\mathbf{x})$는 모든 차원의 기울기를 원소로 하는 벡터로 다음과 같이 정의된다.

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \cdots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right)$$



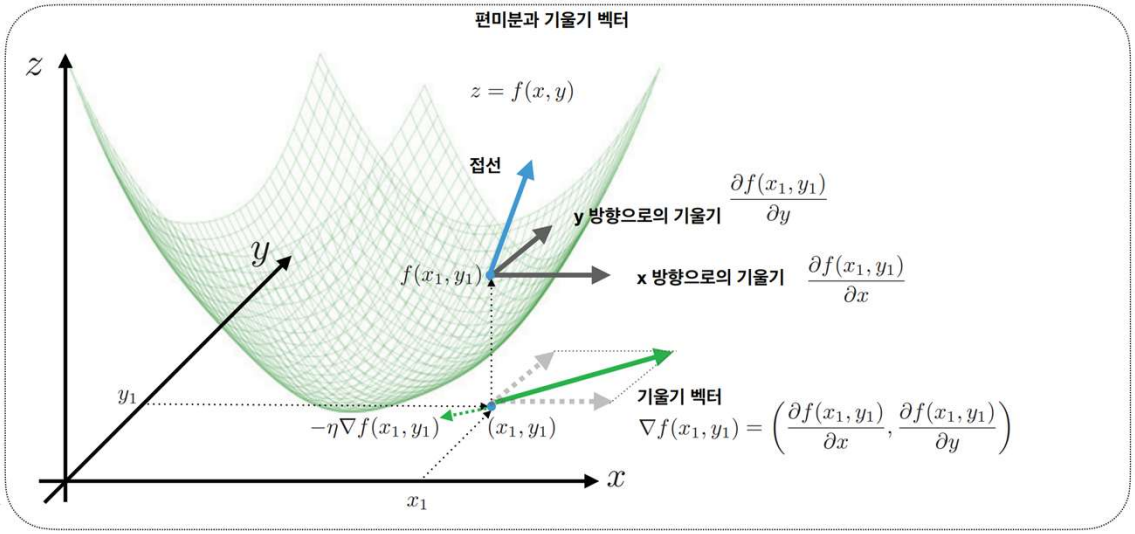
[그림 2-4] 다변수 함수와 경사하강법 적용

2.5 편미분과 기울기

- 두 개의 독립 변수를 가진 경우에는 다음과 같이 두 개의 변수에 대한 편미분으로 정의된다. (x_1, y_1) 지점에서의 기울기 벡터는 다음과 같다.

$$\nabla f(x_1, y_1) = \left(\frac{\partial f(x_1, y_1)}{\partial x}, \frac{\partial f(x, y)}{\partial y} \right)$$

- 기울기 벡터는 그림에서 녹색으로 표시된 화살표이다. 이 화살표는 독립변수로 이루어진 공간에서 정의되고, 최적의 해는 이 기울기 벡터의 반대 방향으로 어딘가에 존재한다.
- 최적의 해를 알 수 없으므로 녹색 점선으로 표현된 것과 같이 작은 값 η를 곱하여 기울기 벡터의 반대방향으로 이동하면 더 작은 목적함수 값을 얻을 수 있다.



[그림 2-4] 다변수 함수와 경사하강법 적용

2.5 편미분과 기울기

- (x_1, y_1)이 최적해에 가까울수록 기울기 벡터는 짧아진다. 기울기 벡터의 크기는 우리가 얼마나 틀렸는지를 가늠할 수 있는 척도가 된다.
- 작은 값 η는 기울기 벡터의 크기를 얼마나 고려하여 이동할지를 결정하는 것으로, **학습률**learning rate이라고 한다.

2.6 모델, 파라미터, 그리고 학습

- 머신러닝은 문제를 해결하는 **모델**model과 이 모델의 동작을 결정하는 **파라미터**parameter, 그리고 이 파라미터를 더 좋은 상태로 변경하는 **학습**learning 동작으로 이루어짐.
- 3x3 해상도의 이미지에서 'ㄱ'과 'ㄴ'을 구분하는 문제
  - 글자의 획이 그어진 부분은 1, 획이 자리를 차지하지 않는 부분은 0의 값을 가진다고 하면 두 글자는 [그림 2-5]와 같이 표현됨.
- **모델**model : 주어진 일을 처리하기 위해 고안된 시스템.
  - 맥락에 따라 작업을 처리하기 위한 프로그램 유형이나, 특정한 입출력이 명시된 프로그램 구조, 학습이 완료된 뒤에 예측을 할 준비가 된 파라미터와 알고리즘을 의미하기도 함.

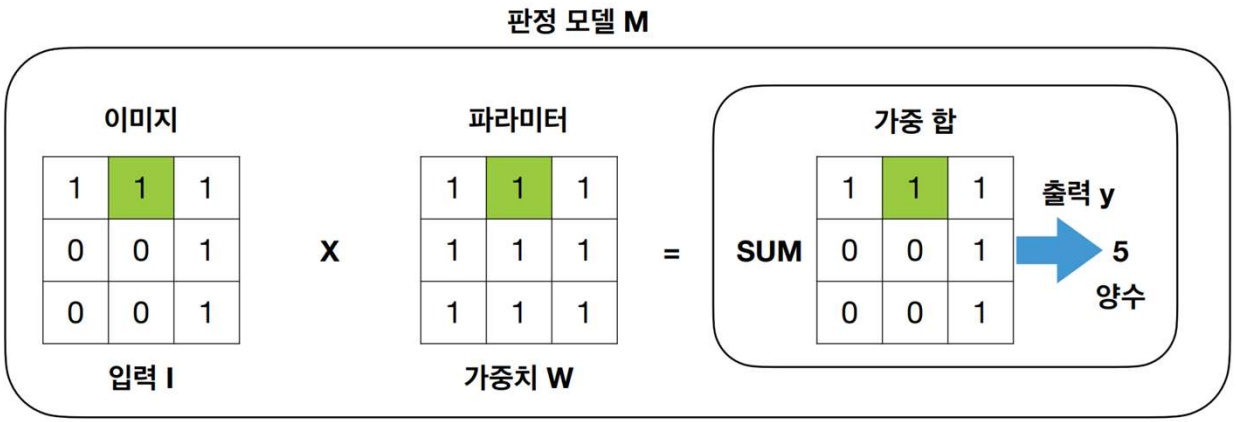
1	1	1
0	0	1
0	0	1

1	0	0
1	0	0
1	1	1

[그림 2-5] 두 자음을 수치 데이터로 표현하는 방법

2.6 모델, 파라미터, 그리고 학습

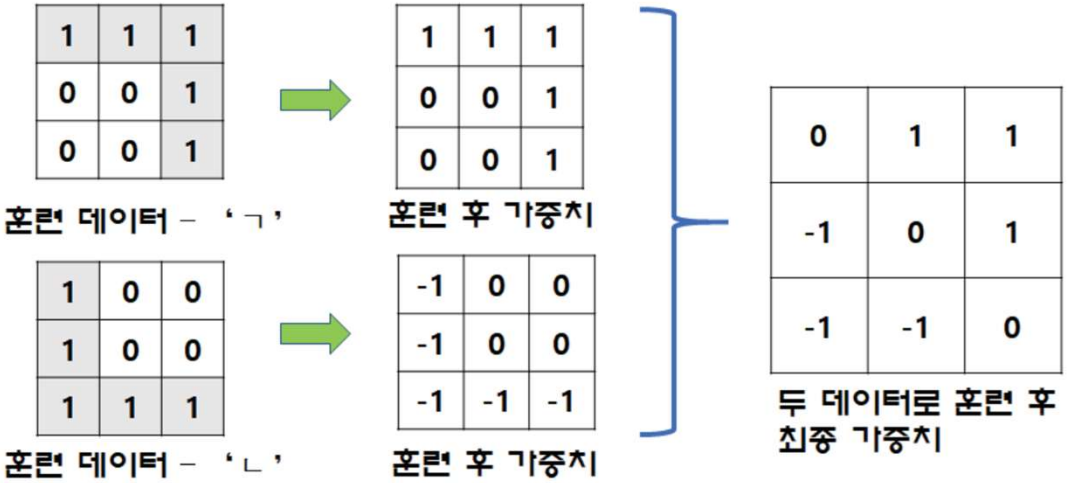
- 모델 M을 다음과 같이 정의하자.
- 입력 I를 받아들이고, 각 입력의 **픽셀**pixel마다 가중치 W를 곱하여 얻은 값을 합산한 뒤, 그 합 y 값이 양수이면 'ㄱ', 음수이면 'ㄴ'으로 판정한다.
- **파라미터** : 가중치 W와 같이 모델의 동작을 결정하는 요소
  - 그림의 가중치는 픽셀에 1이 포함된 어떤 입력에 대해서라도 결과가 양수가 나오기 때문에 무조건 'ㄱ'으로 판정함. 모델의 동작을 바꾸기 위해 가중치 파라미터를 변경할 수 있음.
- 파라미터를 잘 바꾸면 모델이 더 좋은 동작을 하고, 이것은 문제를 해결하는 방법을 **학습**한 것.



[그림 2-6] 두 자음을 구분하는 판정 방법을 가진 모델의 구현 예

2.6 모델, 파라미터, 그리고 학습

- 모델 학습 과정
- 'ㄱ'과 'ㄴ'을 표현하는 입력 데이터를 준비한 뒤, 'ㄱ'을 표현하는 입력 데이터에서 나타나는 1의 위치에 해당하는 가중치는 증가시키고 'ㄴ'을 표현하는 입력 데이터의 1 신호에 해당하는 가중치는 감소시켜 음수가 되도록 함.
- 훈련 이후에 얻은 가중치 파라미터를 가진 모델에 처음 준비했던 두 입력 데이터를 넣으면 'ㄱ'은 3으로, 'ㄴ'은 -3으로 계산되어 제대로 구분할 수 있음.



[그림 2-7] 판정에 사용되는 파라미터를 갱신하는 학습/훈련 방법의 구현

2.6 모델, 파라미터, 그리고 학습

- [그림 2-8]과 같은 입력들을 제공하면 이 입력은 각각 'ㄱ'과 'ㄴ'의 모양인데 모두 0으로 판정되어 구분하지 못함.
- 이와 같이 새로운 데이터에 대해서는 제대로 동작하지 않는 것을 **일반화**generalization 능력이 떨어진다고 말한다.
- 일반화 능력을 높이기 위해 더 많은 훈련 데이터를 사용하거나 모델을 바꾸어야 할 수 있다.

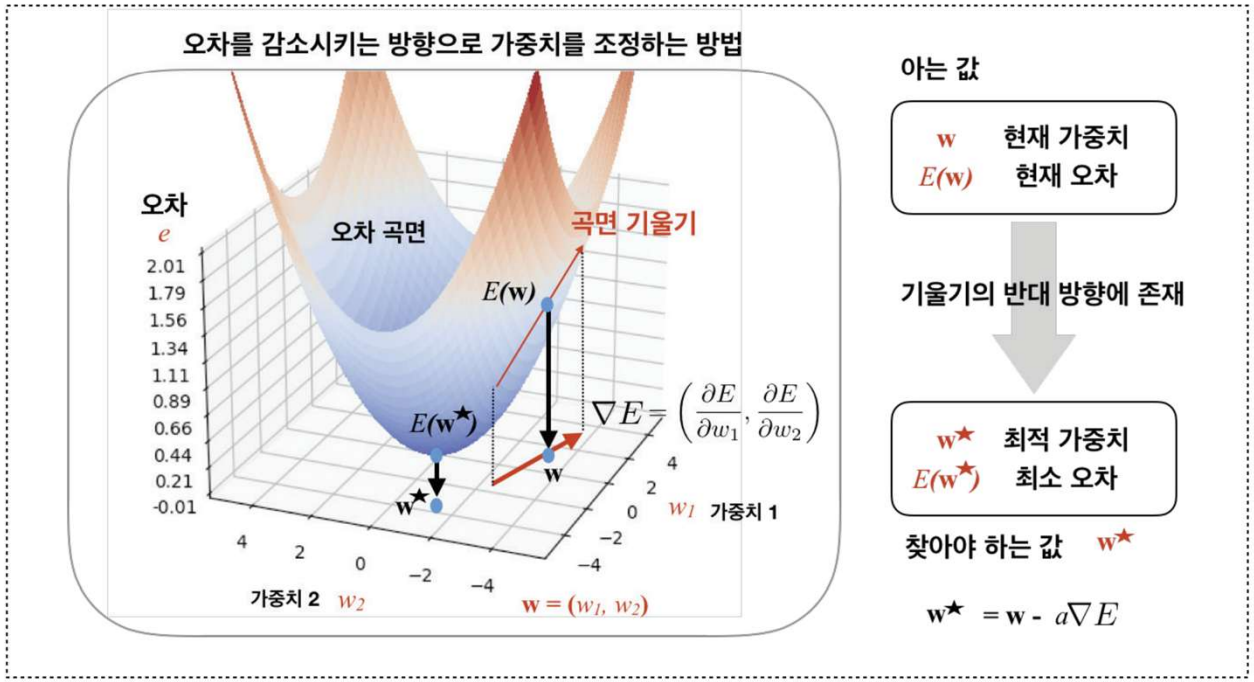
1	1	0
0	1	0
0	1	0

0	1	0
0	1	0
0	1	1

[그림 2-8] 훈련된 모델이 처리할 수 없는 데이터

2.7 오차의 기울기를 이용한 학습의 기본 원리와 최적해

- **오차**error : 모델의 처리 결과와 실제 목표의 차이
- 학습 : 오차가 줄어드는 방향으로 파라미터를 변경하는 일
- [그림 2-9]에서, 모델의 파라미터가 입력에 적용되는 가중치 벡터를 $\mathbf{w}$라고 할 때, 오차는 $E(\mathbf{w})$의 곡면이 됨.



[그림 2-9] 경사하강법에 따른 파라미터 변경과 최적화

2.7 오차의 기울기를 이용한 학습의 기본 원리와 최적해

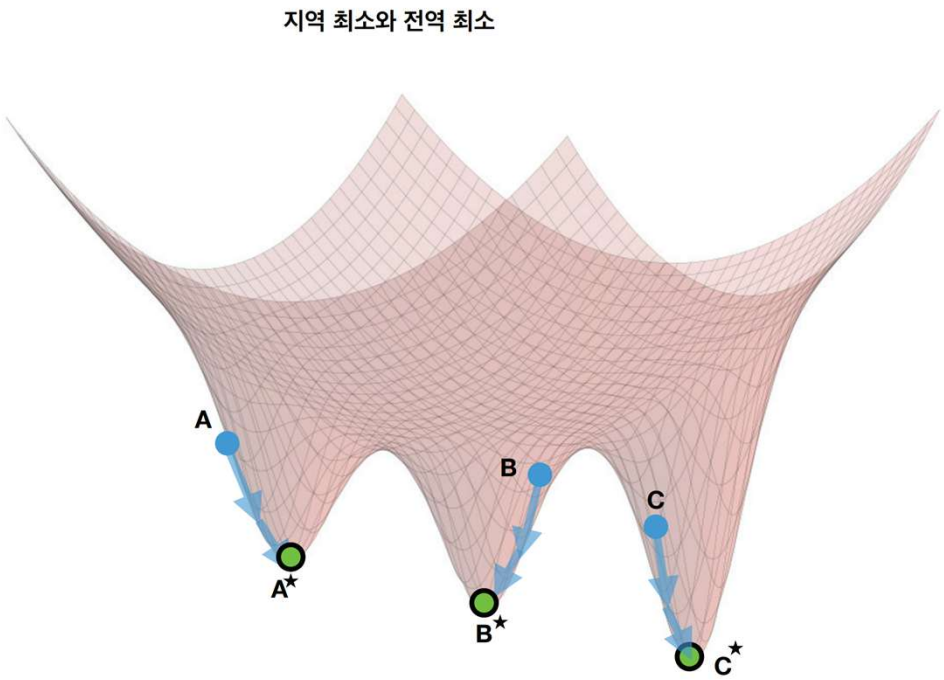
- 오차가 최소가 되는 최적 가중치 $\mathbf{w}^*$를 다음과 같이 표현함.

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}}\, E(\mathbf{w})$$

- **경사 하강법의 핵심 개념**
  - 현재 가중치 $\mathbf{w}$에서 오차 곡면의 기울기를 알면, 낮은 오차 방향으로 이동 가능함
  - 기울기 연산자 ∇(나블라)을 통해 가중치에 대한 오차의 기울기 벡터 ∇E를 계산함
  - 최적 가중치는 ∇E의 **반대 방향**으로 이동하며 점진적으로 접근함
  - 이동 크기 a는 학습률이며, 작은 값으로 반복 갱신함
  - 최적점에 도달하면 기울기 벡터는 0 벡터가 됨

2.7 오차의 기울기를 이용한 학습의 기본 원리와 최적해

- 오차 곡면이 울퉁불퉁한 모양을 갖는다면 경사 하강법을 통해 얻은 답이 전체에서 가장 좋은 답이 아니라, 일정한 영역 내에서 가장 좋은 답일 수 있음.
- 그림에서 A^*나 B^*와 같이 일정한 영역 내에서 가장 좋은 지점을 **지역 최소값**local minimum이라고 하며, 전체 공간에서 가장 좋은 해는 **전역 최소값**global minimum이라고 함.



[그림 2-10] 지역 최소값과 전역 최소값

2.7 오차의 기울기를 이용한 학습의 기본 원리와 최적해

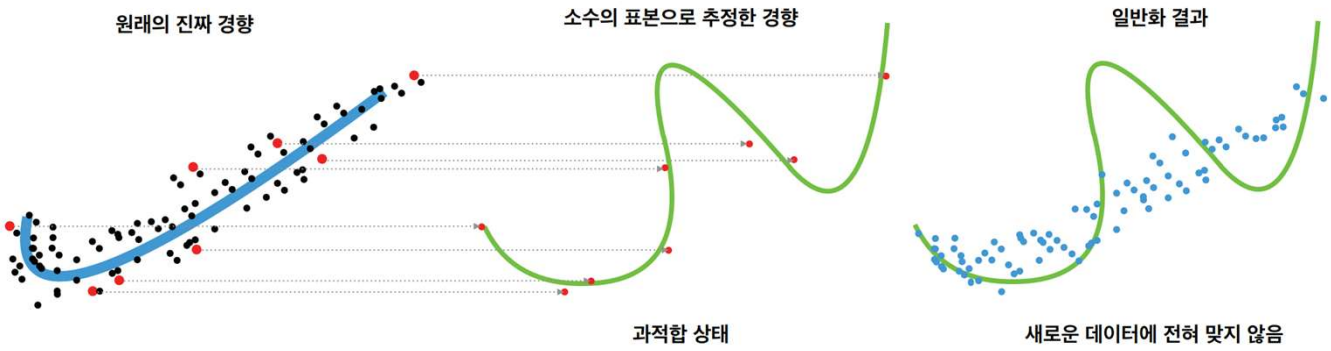
- 경사하강법을 통한 학습은 현재의 가중치 $\mathbf{w}$를 더 좋은 가중치 $\mathbf{w}'$로 바꾸는 일로 오차의 기울기 ∇E를 이용하여 다음과 같이 표현할 수 있음.

$$\mathbf{w}' = \mathbf{w} - \eta \nabla E$$

- 일반적으로 η(eta) 표기를 사용함.
- 이 방법은 언제나 지역 최적값에 붙잡혀 전역 최적값을 찾지 못할 위험이 존재함.
- η는 기울기의 반대 방향으로 얼마나 이동할 것인지 결정하는 값으로, **학습률**이라고 함.
- 이와 같이 모델의 동작을 결정하는 파라미터는 아니지만 학습 과정에 영향을 미치는 값들을 **하이퍼파라미터**hyperparameter라고 함.

2.8 과적합, 과소적합, 그리고 일반화

- **과적합**overfitting : 데이터를 분석한 결과가 특정한 데이터 집합에만 매우 정확하게 일치하고, 다른 데이터에는 잘 들어맞지 않는 상태
- 학습하지 않은 데이터에 모델을 적용하여 값을 예측하거나 분류를 수행하는 등의 데이터 분석을 실시하는 것이 **일반화**generalization인데, 과적합이라는 것은 일반화가 제대로 되지 않는다는 것을 의미함.
- [그림 2-11]과 같이 데이터들을 모두 사용하지 않고 붉은색 점만 학습에 이용하여 경향을 추정하면 가운데 그림을 얻을 수 있음. 그러나 진짜 경향에 따라 만들어진 새로운 점들에 이 모델을 적용하면 새로운 데이터를 전혀 설명하지 못하고, "일반화 역량이 떨어짐"
- 과적합이 일어나는 원인
  - 학습에 사용된 데이터의 수가 너무 적어서
  - 데이터를 설명하기 위해 사용된 곡선이 복잡해서



[그림 2-11] 적은 수의 대표 데이터에 대한 과적합 개념

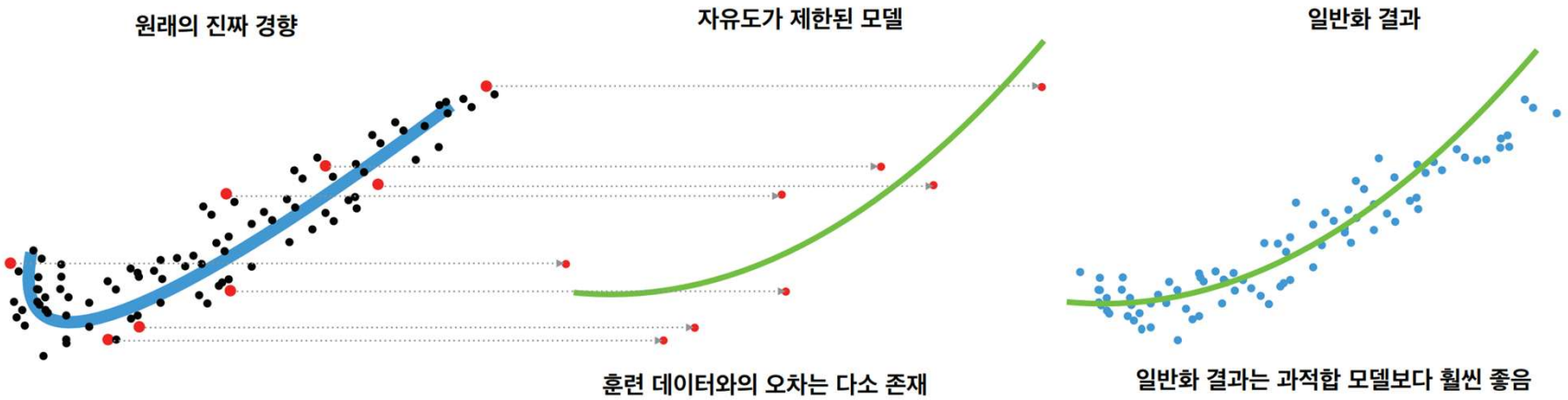
2.8 과적합, 과소적합, 그리고 일반화

- **과소적합**underfitting : 학습이 지나치게 덜 이루어져 새로운 데이터뿐만 아니라 학습 데이터조차 제대로 설명하지 못 하는 모델
- 지나치게 단순한 모델로 학습에 사용되는 훈련 데이터를 설명하려 하면 훈련 데이터에 대해서 큰 오차를 많이 줄이지 못 함.
- 과소적합이 일어나는 이유
  - 모델이 지나치게 단순해서
  - 예측을 제대로 할 수 없는 특징들만 제공되어서

[그림 2-12] 단순한 모델에 의한 과소적합 개념

2.9 과적합을 피하기 위한 방법들

- 과적합의 근본적인 문제는 모델에 지나친 자유를 부여하여 학습에 사용되는 데이터를 설명하기 위해 마음대로 휘어질 수 있기 때문
- 모델이 지나치게 자유롭게 변형될 수 없도록 **규제**나 **정칙화**를 사용



[그림 2-13] 모델에 대한 규제로 과적합 방지 개념

2.9 과적합을 피하기 위한 방법들

- 최적화의 대상인 오차 함수 $E(\mathbf{w})$를 정칙화가 적용된 새로운 함수 $E^r(\mathbf{w})$로 바꿈
  - 추가되는 항 p는 벌칙항

$$E^r(\mathbf{w}) = E(\mathbf{w}) + p$$

- 머신러닝에서 정칙화는 기본적으로 파라미터들이 지나치게 큰 값을 갖지 못 하게 만드는 것
  - 정칙화를 파라미터 **수축**shrinkage이라고 부르기도 함.
- **라소**lasso 기법 : "least absolute shrinkage and selection operator"의 머리글자를 딴 것으로, 벌칙항 p로 l_1 노름을 사용하는 것
  - l_1 노름이 작아지는 방향으로 파라미터를 규제하여 값이 0 쪽으로 수축되게 함.
  - l_1 노름을 사용하기 때문에 **L1 정칙화**라고 부름.

$$E^r(\mathbf{w}) = E(\mathbf{w}) + \alpha\|\mathbf{w}\|_1$$

2.9 과적합을 피하기 위한 방법들

- l_2 노름을 사용한 L2 정칙화도 가능함.
- 통계학에서 **능형 회귀**ridge regression라고 부르는 방법과 같은 개념

$$E^r(\mathbf{w}) = E(\mathbf{w}) + \alpha\|\mathbf{w}\|_2$$

- 인공 신경망에서는 학습 단계에서 신호 연결을 무작위로 끊어 버리는 **드롭아웃**drop-out과 같은 방법을 사용하기도 함(드롭아웃은 8장에서 설명).

	데이터	모델	모델 규제
과적합	데이터의 규모 확대	단순화	강화
과소적합	새로운 특징 확보	복잡화	완화

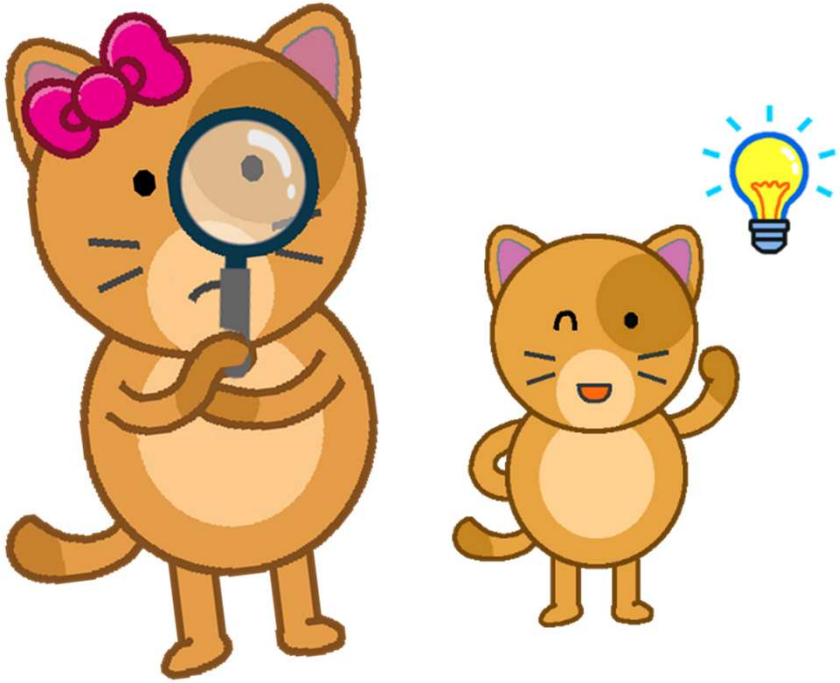
[그림 2-14] 과적합과 과소적합에 대한 대응법

핵심정리 (1)

- 머신러닝을 위해서는 **벡터와 행렬**, **선형대수**, **미적분** 등의 지식이 필요하다.
- 두벡터를 곱할 때는 **아다마르 곱**, **점곱**, **벡터곱** 등을 사용할 수 있다.
- 행렬은 하나 이상의 행과 하나 이상의 열로 된 격자 구조에 수가 담겨 있다.
- 다차원 함수를 **각각의 변수로 편미분**하는 것은 **각 차원의 기울기**를 구하는 일이다.
- 모든 차원으로의 기울기를 구해, 벡터로 표현하면 **기울기 벡터**가 된다.
- **기울기 벡터의 반대 방향**으로 변수를 이동하면 **함수가 최소값이 되는 변수** 쪽으로 접근할 수 있다.
- 기울기를 이용하여 최적의 변수를 찾아가는 방법을 **경사하강법**이라고 한다.
- 함수가 울퉁불퉁한 모양일 경우 경사하강법은 함수 전체에서 가장 작은 값을 갖는 **전역 최소값**이 아니라 일부 영역 내에서 가장 작은 값인 **지역 최소값**에 머물 수 있다.
- 머신러닝 알고리즘은 **모델**, **파라미터**, **학습**의 요소로 이루어진다.

핵심정리 (2)

- 톰 미첼의 머신러닝 정의에 따라 모델, 파라미터, 학습을 설명하면, **모델**은 **작업을 수행하기 위해 설계된 것**으로 **파라미터에 의해 동작**하도록 구현된다. 모델의 작업은 성능 척도에 의해 평가되며, 다양한 데이터를 경험하면서 **평가 점수를 개선하는 방향으로 파라미터를 변경**하는 과정이 **학습**이다.
- 기본적인 **오차 최소화 방법**은 파라미터로 결정되는 오차 곡면 함수의 기울기를 계산하여 이 기울기를 따라 내려가면서 **기울기가 0이 되는 곳을 찾는 것**이다.
- 이렇게 오차가 최소가 되도록 가장 좋은 상태의 파라미를 찾아가는 동작을 **최적화**라고 하며, 학습은 곧 최적화이다. 파라미터를 오차 감소 방향으로 얼마나 옮겨 놓을 것인지, 이러한 학습 동작을 몇 번이나 해야 하는지 등을 제어하는 값들을 **하이퍼파라미터**라고 한다.
- **파라미터**는 모델의 동작을 결정하고, **하이퍼파라미터**는 최적화 과정에 영향을 미친다.
- 모델이 **과적합**하지 않도록 하는 방법은 모델이 데이터에 지나치게 맞춰질 수 없도록 억제하는 것이다.



Questions?

감사합니다.